

P1283R0

Sharing is Caring

Published Proposal, 2018-10-06

This version:

<https://wg21.link/p1283r0>

Author:

[Isabella Muerte](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1283R0](#)

Current Source:

slurps-mad-rips/papers/proposals/sharing-is-caring.bs

Abstract

We need a shared attribute for C++ because some vendors refuse to add their compiler specific attribute syntax into a generalized attribute namespace.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design

4 Wording

4.1 Feature Testing

5 Acknowledgment

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

Generalized attributes were added in C++11 and for a brief period there was hope that all compiler vendors would be able to do away with their custom implementation specific syntax for attributes. Instead, we are still stuck having to use preprocessor macros just so we can have dynamically linked libraries work under multiple compilers. While the standard doesn't say anything about dynamically linked libraries, they do exist (whether we want them to or not). This paper does not seek to define this concept. It just makes a simple attribute required for all compiler vendors, especially those that have not placed their DLL export attribute into a namespace. One particular compiler vendor is extremely guilty of this. While the author doesn't want to name and shame, they will state that the name of the vendor in question starts with the letter "M" and ends in "icrosoft".

This paper is extremely similar to, but not a recreation of, [\[P0276\]](#). While that paper attempted to specify all the possible traits available for visibility in regards to a shared library, this paper focuses entirely on interfaces exported via shared libraries. Additionally, we go one step further and limit this "sharing" to functions only. There are many issues that can arise from sharing a type over program boundaries and this paper is avoiding them. A future paper can argue in favor of expanding what foundation this proposal lays down now.

Note: The author is more than willing to pull this paper if every vendor would just behave and promise to namespace their compiler specific attributes so we don't have to go through something like this in the future.

§ 3. Design

The name `shared` is recommended as it involves sharing interfaces across program boundaries. This differs from [P0276] which used GCC's existing `visibility` name as a base. There is no intended "import" vs "export" function declaration dichotomy. Platforms that might have required a difference are no longer supported by their vendors, and have instead added support for a single export.

Note: These same platforms may still require an "import" vs "export" for non-function entities.

§ 4. Wording

Wording is relative to [N4762]

9.11.12 Shared attribute [dcl.attr.shared]

¹The *attribute-token* `shared` specifies that an entity with external linkage [`basic.link`] may be obtained:

- (1.1) by current translation unit at program startup from a library file if the entity has no definition in current translation unit. [Note: attribute `shared` only provides information that the entity is available to obtain at runtime from a library file. – end note]
- (1.2) by other translation units at runtime or at program startup if the entity has a definition in the current translation unit. [Note: In this case, `shared` attribute prevents optimizing compilers from removing or "stripping" the entity from the program and from changing the entity type or signature. – end note]

²`shared` shall appear at most once in each *attribute-list* and no *attribute-argument-clause* shall be present. The attribute may be applied to the *declarator-id* in a non-inline function.

³The first declaration shall specify the `shared` attribute if any other declaration specifies the `shared` attribute. If an entity is declared with the `shared` attribute in one translation unit and the same entity is declared without the `shared` attribute in another translation unit, the program is ill-formed.

§ 4.1. Feature Testing

Tests for the `shared` attribute can be completed via the current convention of using `__has_cpp_attribute` like so: `__has_cpp_attribute(shared)`.

§ 5. Acknowledgment

Special thanks to Antony Polukhin, who did a majority of the heavy lifting last time a paper like this was brought before the committee.

§ References

§ Informative References

[N4762]

Richard Smith. [Working Draft, Standard for Programming Language C+](https://wg21.link/n4762). 7 July 2018. URL: <https://wg21.link/n4762>

[P0276]

Antony Polukhin. [A Proposal to add Attribute `\[\[visible\]\]`](https://wg21.link/p0276). URL: <https://wg21.link/p0276>